SimpleStoreServiceOrchestration

System Design Description

Eclipse Arrowhead 5.2.2 | core system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document describes implementation ownership, architecture, persistence, security and operational boundaries. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple. 1 detailed Java SysDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-core-go/serviceorchestration-simple.

SimpleStoreServiceOrchestration - Go 5.2.2

Standalone core system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/SimpleStoreServiceOrchestration. Entry point: ./cmd/serviceorchestration-simple. Module: arrowhead/ah5-core-go/serviceorchestration-simple. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. Existing role selection and API behavior are retained. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8442 (Arrowhead default). The shipped config.json sets "listen": "0.0.0.0:8442". Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../../LISTENING-PORTS.md) for precedence and all systems.

Detailed implementation evidence

0 native Go SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Native compatibility profile

Compatibility profile and remaining work

This document is deliberately separate from the route inventory. It records the actual implemented behavior and material gaps, rather than calling a set of HTTP handlers a complete Java replacement.

Implemented

- Authentication: PASSWORD identities, bcrypt cost 10, batch management, credential changes, credential-authenticated logout, UUID sessions, one active session per identity, expiration checks, session query/closure and identity query filters. Credentials and session tokens are excluded from management responses. Credential changes and management updates invalidate sessions (a stricter security choice).

- ConsumerAuthorization: provider and MGMT policy levels, MGMT precedence, ALL/WHITELIST/BLACKLIST/SYS_METADATA policies, scoped/default fallback, all-scopes checks, grant/lookup/query/revoke/check operations. Missing policies deny access.

- Authorization tokens: usage/time-limited tokens; URL-base64 self-contained tokens with the Java pipe-separated field order; RS256/RS512 JWTs with psn/csn/ccn/tat/tan/sco claims; public-key export; management generation/query/revocation; provider-bound online verification with atomic usage decrement. Raw tokens are returned only at generation, not retained for later retrieval.

- Encryption-key registration: Java AES/ECB/PKCS5Padding and AES/CBC/PKCS5Padding wire encodings, provider-specific IVs for CBC, owner/management removal. Provider keys are authenticated-encrypted at rest with AES-GCM and provider identity as additional authenticated data. The server certificate's RSA key signs JWTs.

- Simple orchestration: consumer-specific store, unique priorities per consumer/service, priority modification, preferred providers, ONLY_PREFERRED, per-service MATCHMAKING, ignored-field/flag warnings, pull/history.

- Dynamic orchestration: service discovery through ServiceRegistry, mandatory authorization checks, preferred providers, interface operations, token acquisition, MATCHMAKING, provider-consented exclusivity capped by allowExclusivity, atomic lock conflict rechecks, lock management, pull/history.

- Both orchestrators: persisted subscriptions, owner checks, management subscription/query/unsubscribe/trigger operations, persisted jobs and HTTPS push callbacks. Callback TLS is pinned to the target system identity before sending results. Interrupted in-progress jobs become ERROR on restart instead of being redelivered automatically.

- General management: sanitized configuration map, bounded persisted audit log querying, authenticated echo. Source-generated service descriptions and opt-in periodic service registration.

- Shared runtime: mandatory local-cloud mTLS, identity-token certificate binding, fixed authority identity pinning before credentials, redirect rejection, coalesced login refresh, conditional token invalidation, body limits, server timeouts, atomic JSON snapshots and exclusive process ownership of a state file.

- ServiceRegistry: pre-existing Go implementation plus corrected AuthorizationVerifyRequestDTO fields/operation names and removal of an implicit Sysop-name privilege escalation in outsourced token verification. Its original configuration and publisher are preserved.

Not implemented / not equivalent

1. MQTT APIs (since 2026-10-24) differ from Java in three deliberate points: the response topic must lie under arrowhead/response/<requester>/, requests with an invalid identity token are not answered (no 401 response), and identity login/logout/change stay HTTPS only. Request topics, envelopes, operation names and error payloads follow the Java MQTT API; see "Changes 2026-10-24 (MQTT)". 2. Dynamic federation, translation and QoS integration are absent. Requests that enable ALLOW_INTERCLOUD, ONLY_INTERCLOUD or ALLOW_TRANSLATION, or supply QoS requirements, fail explicitly with 501. They are not silently treated as successful local equivalents. Foreign-consumer management token generation also returns 501. This is unsuitable for a federated orchestration replacement until those paths are ported and tested. 3. Blacklist support and authorization-driven management filters are not ported to the four new systems. Management uses identity sysop privileges or an explicit whole-management whitelist. This is less granular than per-operation authorization; do not use the whitelist as a substitute for fine-grained policy enforcement. 4. The new systems do not yet publish native sysMonitor/AMQP telemetry. Their echo APIs and history/logs exist, but they will not satisfy a LocalCloudManager native-telemetry readiness requirement. The existing ServiceRegistry publisher remains available. 5. No Java SQL database compatibility/import. Each new system uses a private atomic JSON state file, not JPA/MySQL. Never mount a Java database volume as a Go state file. Java identity, authorization, key, store, lock, subscription and history data need a separately reviewed importer before migration. 6. Configuration is JSON, not Spring properties, environment override semantics, keystores or Java's full property catalog. get-config returns a sanitized native configuration subset in the Java map response wrapper. There is no cloud registration health/readiness gate; registration failures are retried and reported to the process log. 7. Validation/normalization and query semantics are not exhaustively matched. Supply canonical names and enum

spelling. Field lengths, case normalization, HTTP error details/statuses, empty/null distinctions, ordering/default page sizes and warning text still need Java golden-contract tests. Pagination is bounded, but is not a full port of Java's per-entity paging service. JSON inputs use dynamic documents rather than generated, exhaustive DTO validation. 8. Metadata supports the documented comparisons and dotted/indexed paths, but REGEXP uses Go RE2, not Java Pattern; lookarounds/backreferences are not supported. Mixed-type numeric/collection edge cases need further Java fixture tests. 9. Dynamic candidate ranking is deterministic local/preferred ordering, not the complete Java matchmaking/QoS/fallback pipeline. Scopes are checked fail-closed; a denied requested operation can reject orchestration instead of yielding Java's precise partial-result behavior. Token acquisition is not rolled back if a later lock conflict aborts orchestration. 10. Push uses one worker and one delivery attempt, without Java's Quartz/thread-pool behavior, HTTP/plain MQTT callbacks, retries, or automatic event-driven re-orchestration. HTTPS result-delivery behavior is tested; duplicate delivery avoidance does not constitute exactly-once delivery. Notifications interrupted by a crash are reported as uncertain ERROR. 11. Logs are native bounded audit records (1,000 retained), not the full Java logging database or JVM-specific exceptions. Sessions/tokens/jobs/subscriptions/locks are validated for expiry at access but do not have complete Java cleaner/retention policies; state can grow over time. Expired entities may differ in query visibility. 12. No certificate revocation enforcement beyond Go's ordinary certificate-chain/time/hostname checks. No blacklist, rate limiter, distributed transactions, HA, distributed locks, SQL migration, or production security certification. Offline self-contained token revocation requires provider-side expiration/revocation handling; online revocation does not invalidate already-issued offline tokens.

Release gate

Do not describe this profile as fully interchangeable with the Java cloud. Before replacement: implement the required missing capabilities, compare successful and failing requests against Java for every DTO/operation, verify all discovery/security policies, migrate data with rollback, verify monitoring/UI readiness and federation, and rerun the cloud security evaluation. None of those deployment changes is made by this source delivery.

Changes 2026-10-17 (GoCloud deployment)

- Dynamic orchestration passes each requested operation as an authorization scope; a provider qualifies only if

every requested operation is granted. Without operations the all-scopes check applies (as before). Test: TestNativeServiceRegistryIntegration.

- Deployed in Arrowhead-test-cloud-go (see its GO-NATIVE-FULL-2026-10-17.md) by operator decision, with the gaps

listed above accepted; the release gate above is not thereby satisfied.

- Inter-cloud orchestration: ALLOW_INTERCLOUD (fallback when no local provider) and ONLY_INTERCLOUD through the local

Gatekeeper (gatekeeperURL): globalServiceDiscovery query, interCloudNegotiation negotiate, GateTunnel interface result. ALLOW_TRANSLATION and QoS requirements still return 501.

- Native sysMonitor telemetry (monitorEnabled, amqpURL): sysMonitor offer on generic_amqps, echo/power/status.

- managementPolicy: authorization: ConsumerAuthorization decides management access per service and operation.

- Requested operations match string-set operations (MQTT/AMQP templates) as well as HTTP operation maps.

- Push subscriptions store notifyRecipient: a non-sysop management subscriber (orchestration by proxy) receives

the result; own and sysop subscriptions deliver to the target.

Changes 2026-10-24 (translation)

- ALLOW_TRANSLATION: an interface is translatable when it does not match the template or property requirements

natively (as Java assortInterfacesAndMarkIfNonNative), so a bridge can also translate only the data model of the accepted template (generic_https senml -> generic_https simpleJson). Native candidates still win; the consumer's authorization for the provider is still checked before a bridge is built.

Changes 2026-10-24 (MQTT)

- MQTT API of all five core systems (internal/core/mqtt.go, ServiceRegistry/mqtt.go), the Java mqtt.api.enabled

counterpart: the Java request envelope (MqttRequestTemplate: traceId, authentication IDENTITY-TOKEN//<token>, responseTopic, qosRequirement, params, payload) on the Java base topics + operation (arrowhead/serviceregistry/{device-discovery,system-discovery,service-discovery,management,general/management, monitor}/, arrowhead/authentication/identity/[management/], arrowhead/consumer-authorization/authorization[-token]/ [management/], arrowhead/serviceorchestration/orchestration/[management/{push,history,lock,store,plant-rules}/], general/management/, monitor/echo); the response (MqttResponseTemplate: status, traceId, receiver, payload; errors as ErrorMessageDTO with exceptionType and origin = topic) goes to the request's response topic.

- The adapter verifies the identity token and runs the request in process through the HTTPS route, so the blacklist,

management (whitelist/authorization) checks and handlers are the same as over HTTPS. Path parameters come from params (a single one also from a string payload, as Java), the other params become the query, a list payload of a removal operation is its list parameter (names, instanceIds, uuids, ids, tokenReferences, systemNames, serviceInstances, keys).

- Response binding (Go): the response topic must lie under arrowhead/response/<system of the token>/; with a broker

ACL that lets each client read only its own prefix, a stolen token cannot be used to read answers.

- Configuration: mqttURL (ssl://host:port, client certificate = system certificate) and mqttTopicPrefix (a second

orchestrator on the same broker needs another prefix); ServiceRegistry mqtt.api.enabled, mqtt.broker.address, mqtt.broker.port, mqtt.topic.prefix (requires TLS and the outsourced policy). With MQTT enabled every service is also registered with a generic_mqtts interface (baseTopic, operation list), as Java.

- ServiceRegistry: the Blacklist filter never checks Blacklist itself (Java BlacklistFilter); a Java Blacklist, which

registers before it serves its check, was refused under force.blacklist.filter.

- cmd/mqtt-call: command-line client for one MQTT API request (tests).

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-core-rust/serviceorchestration-simple.

SimpleStoreServiceOrchestration (Rust)

Arrowhead 5.2.2 core system. src/main.rs is built as binary serviceorchestration-simple by the arrowhead-core-rust crate in ... serviceorchestration-simple.json and documentation/ link to ../examples and ../documentation/SimpleStoreServiceOrchestration.

Detailed implementation evidence

1 native Rust SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-SysDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-core-rust/documentation/SimpleStoreServiceOrchestration/SimpleStoreServiceOrchestration-SysDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

SimpleStoreServiceOrchestration

System Design Description

Abstract
This document describes the design and implementation of the SimpleStoreServiceOrchestration core system of Arrowhead 5.2.2 in Java Spring (`ah5-core-java-spring/serviceorchestration-simple`).

Contents

1 Overview **3**

2 Implementation **4**
2.1 Implementation language and tools 4
2.2 Functional properties implementation 4
2.3 Non functional properties implementation 5
2.4 Deployment 5
2.5 Local modifications 5
2.6 Verification 6

3 Services **7**

4 References **7**

5 Revision History **9**
5.1 Amendments 9
5.2 Quality Assurance 9

1 Overview

This document describes the SimpleStoreServiceOrchestration system implementation, which realizes the SimpleStoreServiceOrchestration SysD [1]. SimpleStoreServiceOrchestration orchestrates from a static store: an operator records which service instance each consumer shall use, with a priority, and the orchestrator returns these rules on request. It offers the same `serviceOrchestration` service as DynamicServiceOrchestration, plus management of the store. In Section 2, we describe implementation details of the system.

2 Implementation

This implementation is based on the SysD document SimpleStoreServiceOrchestration [1], the upstream Eclipse Arrowhead Java Spring implementation 5.2.2 [2] and the local changes listed in Section 2.5.

2.1 Implementation language and tools

- Java 21, Spring Boot 3.4.9 (web, security, data JPA, Quartz), springdoc-openapi 2.8 (Swagger UI), Log4j 2.
- Maven module `serviceorchestration-simple` of `ah5-core-java-spring`; it depends on `arrowhead-common-utils` and `arrowhead-data-transfer-objects` 5.2.2 from the local `ah5-common-java-spring` (Eclipse Paho MQTT v3 1.2.5, RabbitMQ AMQP client 5.22.0).
- Database: MariaDB/MySQL (MariaDB Connector/J; MySQL 8.4 server in the test cloud), schema validated by Hibernate (`ddl-auto=validate`); SQL scripts in `src/main/resources/database`.
- Stateless request processing; all state is in the database. Scheduled jobs use Quartz.

2.2 Functional properties implementation

The Java packages below `eu.arrowhead` implement the system:

Package	Responsibility
`api.http, api.mqtt`	Controllers and topic handlers of the five services.
`service.utils.Service Orchestration`	Store-based orchestration (priorities, preferred providers, matchmaking).
`service.thread`	Push orchestration thread and workers.
`service, service.validation, service.normalization`	Orchestration, store, push and history logic.
`jpa, quartz`	Store, subscription, job and log entities; cleaner job.

Every request passes the common filter chain: authentication filter (by `authentication.policy`), optional blacklist filter, management service filter (for management services), then the controller, validation and normalization, the service logic and the database service. The same logic is reachable through the MQTT topic handlers.

2.2.1 Data handled and stored

Database `ah_serviceorchestration_simple`:

Tables	Content
`orchestration_store`	Store entries: consumer, service definition, instance, priority.
`subscription`	Push subscriptions.
`orchestration_job`	Pull and push jobs.
`logs`	Log entries for `get-log`.

The data models of the interfaces are the Java records of `eu.arrowhead.dto` (common library); the IDD documents name them without the `DTO` suffix. The records used by this system are: `Authorization TokenGenerationResponseDTO`, `KeyValuesDTO`, `LogEntryDTO`, `LogEntryListResponse DTO`, `LogRequestDTO`, `MetadataRequirementDTO`, `OrchestrationHistoryQueryRequestDTO`, `OrchestrationHistoryResponseDTO`, `OrchestrationJobDTO`, `OrchestrationNotifyInterface DTO`, `OrchestrationPushJobListResponseDTO`, `OrchestrationPushTriggerDTO`, `Orchestration RequestDTO`, `OrchestrationResponseDTO`, `OrchestrationResultDTO`, `OrchestrationService`

`RequirementDTO`, `OrchestrationSimpleStoreListRequestDTO`, `OrchestrationSimpleStoreListResponseDTO`, `OrchestrationSimpleStoreQueryRequestDTO`, `OrchestrationSimpleStoreRequestDTO`, `OrchestrationSimpleStoreResponseDTO`, `OrchestrationSubscriptionListRequestDTO`, `OrchestrationSubscriptionListResponseDTO`, `OrchestrationSubscriptionQueryRequestDTO`, `OrchestrationSubscriptionRequestDTO`, `OrchestrationSubscriptionResponseDTO`, `PageDTO`, `PriorityRequestDTO`, `QoSRequirementDTO`, `ServiceInstanceInterfaceResponseDTO`.

2.3 Non functional properties implementation

2.3.1 Security

TLS and client authentication are configured with the Spring `server.ssl.*` properties (PKCS#12 keystore and truststore). The authentication filters of the common library implement the four authentication policies; `TokenCertificateBinding` binds identity tokens to the client certificate and `CrlRevocationConfig` enables CRL checking in Tomcat (both local). The private key of the keystore is also used to sign the system's outgoing requests where required.

2.3.2 Power management

None; the system is a server process without power states. The `sysMonitor` `power` observation always reports line supply.

2.3.3 Internal monitoring

Log4j 2 logs to stdout and rolling files; log entries are also written to the `logs` table and served by `generalManagement.get-log`. The native `sysMonitor` publisher (`eu.arrowhead.common.sysmonitor`) publishes `echo`, `power` and `status` every 30 s.

2.3.4 Configuration

Configuration is read from `application.properties` and environment variables (Spring relaxed binding, e.g. `SERVICE_DISCOVERY_POLICY`); secrets come from files through `SPRING_CONFIG_IMPORT=configtree:/run/secrets/`. The properties are listed in the SysD. `generalManagement.get-config` returns the public ones.

2.4 Deployment

The test cloud (`Arrowhead-test-cloud-java`) builds the image `arrowhead-serviceorchestration-simple:5.2.2-sysmonitor` from the released image `aitiaiiot/arrowhead-serviceorchestration-simple:5.2.2` with the locally built `arrowhead-serviceorchestration-simple-5.2.2.jar` (`java-patch/Dockerfile`). The container runs read-only with the configuration baked into `/app/config`, only its keystores mounted, and publishes port 8442 on the loopback address of the host. Its database runs in a separate container.

2.5 Local modifications

Compared to upstream 5.2.2, this workspace changes:

- Startup is refused when the effective configuration is below the secure profile (`SecurityProfileGuard`).
- Port 8442 instead of 8465.
- The HTTP `monitor` service was removed; telemetry is the native `sysMonitor` service.
- Token-to-certificate binding, CRL checking, MariaDB driver.

2.6 Verification

The module has 21 JUnit test classes; the regression report set recorded on 2026-10-20 covers 316 tests without failures. Live-cloud smoke tests are a separate verification step; the plant-management migration is not claimed as deployed by this update.

3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`serviceOrchestration`	[1]	[3]	[4], [5]
`serviceOrchestrationSimple StoreManagement`	[1]	[6]	[7], [8]
`serviceOrchestrationPush Management`	[1]	[9]	[10], [11]
`serviceOrchestrationHistory Management`	[1]	[12]	[13], [14]
`generalManagement`	[1]	[15]	[16], [17]
`sysMonitor`	[1]	[18]	[19]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
serviceDiscovery	[20]	[21]	–
identity	[22]	[23]	–
authorization	[24]	[25]	–
blacklistDiscovery	Blacklist	–	–
notify interface	Subscribers	–	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

4 References

[1] J. Delsing, “SimpleStoreServiceOrchestration – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-SysD.pdf.

[2] AITIA International, “Eclipse arrowhead 5 core systems (java spring) and documentation,” https://aitia-iiot.github.io/ah5-docs-java-spring/, 2025.

[3] J. Delsing, “serviceOrchestration – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestration-SD.pdf.

[4] ——, “serviceOrchestration HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestration-HTTP-IDD.pdf.

[5] ——, “serviceOrchestration MQTT/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestration-MQTT-IDD.pdf.

[6] ——, “serviceOrchestrationSimpleStoreManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationSimpleStoreManagement-SD.pdf.

[7] ——, “serviceOrchestrationSimpleStoreManagement HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationSimpleStoreManagement-HTTP-IDD.pdf.

[8] ——, “serviceOrchestrationSimpleStoreManagement MQTT/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationSimpleStoreManagement-MQTT-IDD.pdf.

[9] ——, “serviceOrchestrationPushManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-SD.pdf.

[10] ——, “serviceOrchestrationPushManagement HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-HTTP-IDD.pdf.

[11] ——, “serviceOrchestrationPushManagement MQTT/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-MQTT-IDD.pdf.

[12] ——, “serviceOrchestrationHistoryManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-SD.pdf.

[13] ——, “serviceOrchestrationHistoryManagement HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-HTTP-IDD.pdf.

[14] ——, “serviceOrchestrationHistoryManagement MQTT/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-MQTT-IDD.pdf.

[15] ——, “generalManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-generalManagement-SD.pdf.

[16] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-generalManagement-HTTP-IDD.pdf.

[17] ——, “generalManagement MQTT/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-generalManagement-MQTT-IDD.pdf.

[18] ——, “sysMonitor – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-sysMonitor-SD.pdf.

[19] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[20] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[21] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[22] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[23] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, authentication-v5.2.2-identity-SD.pdf.

[24] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, consumerauthorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

[25] ——, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorization-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-07	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 SimpleStoreServiceOrchestration

1.1 System overview

Native Rust implementation of the local Java module `serviceorchestration-simple/`. Each named executable checks its configured role and owns an independent identity, listen port and state file. Shared Rust modules implement the runtime; no Java, Python or Go process implements the service.

1.2 Produced and consumed services

Produced HTTPS definitions: `generalManagement`, `serviceOrchestration`, `serviceOrchestrationHistoryManagement`, `serviceOrchestrationPushManagement`, `serviceOrchestrationSimpleStoreManagement`. Service versions are 1.0.0; the target system version is 5.2.2. Authorities consumed as applicable are Authentication, ServiceRegistry and ConsumerAuthorization. Explicit pinned HTTPS origins and valid cloud identities must be provisioned.

1.3 Architecture and persistent state

The listener uses Tokio, Hyper/Axum and rustls with mandatory client certificates. Verified peer identity is supplied through a private request extension, never a declared-identity header. Business modules share locked JSON state with clone/validate/fsync/rename transactions. Credentials and keys are separate owner-only files. Cross-system operations and callbacks are not distributed transactions.

1.4 Security

Local cloud CN/profile, certificate chain and validity are verified. Identity tokens are checked against Authentication and bound to the client certificate. Management requires a verified sysop or explicit broad whitelist. Outbound peer chain/SAN and expected system CN/profile are checked before credentials are sent. Optional PEM CRLs are enforced at startup-configured trust boundaries and require process restart to reload. No plaintext HTTP listener, TLS bypass or declared SYSTEM identity mode is provided.

1.5 Implementation status and delimitations

This is a development profile, not a certified interchangeable Java replacement. HTTP route presence is not semantic equivalence. Java database migration, MQTT application APIs and production qualification remain outstanding. Stored candidate priority selection, preferred providers, matchmaking, history, subscriptions and explicit HTTPS push jobs are implemented. Store results intentionally retain Java's sparse result shape. Full Java query/priority-normalization semantics, event-driven reevaluation, MQTT callbacks and Java state import are not ported. Unsupported simple-store criteria are reported as ignored warnings.

2 Implementation structure

See `src/security.rs`, `src/store.rs`, `src/web.rs` and the role-specific module. Transactions clone state and publish only after durable rename. State locking prevents concurrent processes from sharing the same path. A directory-sync failure after rename is an uncertain durable commit, not an automatic rollback. HTTPS push callbacks are at-least-once across crash recovery; downstream effects can require reconciliation.

2.1 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/serviceorchestration/general/mgmt/get-config	management
POST	/serviceorchestration/general/mgmt/logs	management
POST	/serviceorchestration/orchestration/pull	identity
POST	/serviceorchestration/orchestration/subscribe	identity
DELETE	/serviceorchestration/orchestration/unsubscribe/{id}	identity
POST	/serviceorchestration/orchestration/mgmt/push/subscribe	management
POST	/serviceorchestration/orchestration/mgmt/push/trigger	management
DELETE	/serviceorchestration/orchestration/mgmt/push/unsubscribe	management
POST	/serviceorchestration/orchestration/mgmt/push/query	management
POST	/serviceorchestration/orchestration/mgmt/history/query	management
POST	/serviceorchestration/orchestration/mgmt/simple-store/create	management
POST	/serviceorchestration/orchestration/mgmt/simple-store/query	management
PUT	/serviceorchestration/orchestration/mgmt/simple-store/modify-priorities	management
DELETE	/serviceorchestration/orchestration/mgmt/simple-store/remove	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.